

The Independent EECS Program

A self-directed path through computer science, computer engineering, and software engineering — benchmarked against MIT Course 6-3 and extended to MEng depth. Math, writing, breadth, and the skill of learning itself are built into the path.

How to use this document

This is a path, not a reference. Read it top to bottom and do the blocks in order.

- The program is **5 prerequisite blocks**, **30 numbered blocks**, and **5 always-on habits**. A numbered block is one course-sized unit of work (~150–200 hours).
- Every block has the same shape: **Why** · **Do** (one primary resource) · **Build** · **Done when** · **Hours**.
- There is **exactly one primary resource per block**. Alternatives are in Appendix A. Don't open Appendix A unless the primary genuinely isn't working after two honest weeks.
- ✓ **marks a book you already own**. The shelf table below shows where each one goes.
- **Full-time** (~45 hrs/wk): 3–4 blocks per term, four years plus an optional fifth "MEng" year. **Part-time** (~20 hrs/wk): 2 blocks per term, about seven years. Both schedules in Part 5.
- Print the checklist in Appendix G. Put it on the wall. Then do "Your first week" at the very end of this document.

The five rules (expanded in Appendix B, justified in P1):

1. No lecture without its problem set the same week.
2. A block is done when the *Done when* line is true. Not before.
3. Never start a new block until the current one is done.
4. Math and writing are daily habits and are never paused.
5. Keep a daily log, in git.

One correction on the target: CSAIL is MIT's research lab, not a degree. The degree is **Course 6-3** (SB in Computer Science and Engineering); the five-year version adds an **MEng** with a thesis. This program targets 6-3 + MEng depth and exceeds the real 6-3 chart on math, systems, and writing.

Your shelf — where each owned book is used

✓ Book	Used in	How
Oakley, <i>A Mind for Numbers</i>	P1	The primary text for learning how to learn. Read it before anything else.
Velleman, <i>How to Prove It</i>	P3 , then Block 10	Proof-writing primary. Chapters 1–3 in Phase 0; 4–7 during Math for CS.
Lockhart, <i>Arithmetic</i>	P3	Alongside Velleman. Also the best warm-up for binary representation in Nand2Tetris.
Ward, <i>How Linux Works</i> , 3e	P5 , Block 6 , reread before Block 16	Chapters 1–7 with Missing Semester; 8–17 during C; the whole thing before OS.
Courant & Robbins, <i>What Is Mathematics?</i>	Year 1 , companion	One chapter every two weeks alongside calculus: 1–2 in fall, 6–8 in spring, 3–5 with Block 10.
Strogatz, <i>The Joy of X</i> · Ellenberg, <i>Shape</i>	Year 1 , pleasure reading	Evenings. They keep math feeling like something you'd choose.
Feynman, <i>Six Easy Pieces</i>	Block 3	Companion to Physics I.
Justice, <i>How Computers Really Work</i>	December of Year 1	Hardware half right before Nand2Tetris.
Kernighan & Ritchie, <i>The C Programming Language</i>	Block 6	Primary. Every exercise.
Zingaro, <i>Algorithmic Thinking</i> , 2e	Block 6 → Block 13	First half as C practice; second half as algorithms practice.
Bryant & O'Hallaron, <i>CS:APP</i> , 3e	Block 9	Primary. The most important book you own.

✓ Book	Used in	How
Ball, <i>Writing an Interpreter in Go</i>	Block 12	Replaces the first half of Crafting Interpreters.
Christian & Griffiths, <i>Algorithms to Live By</i>	Block 13	Pleasure reading during Algorithms I.
Hopcroft, Motwani & Ullman, <i>Automata Theory</i>	Block 24	Primary for Theory of Computation. Sipser is the gentler fallback.
Weir, Brown, Yarros	Habit 5	Nightly pleasure reading. Weir especially — engineering-as-narrative — counts.

The map

Term	CS / CE blocks	Math	Science / Breadth	Reading this term	Milestone
Phase 0 (0–5 mo)	P1 learn to learn · P2 read/think/write · P4 on-ramp · P5 tooling	P3 prereqs	—	✓ Mind for Numbers · <i>Make It Stick</i> · Adler · Pólya · ✓ How to Prove It 1–3 · ✓ Arithmetic · ✓ How Linux Works 1–7	Phase 0 exit tests
Y1 Fall	1 · CS61A	2 · Calculus I	3 · Physics I (✓ Six Easy Pieces)	Williams, <i>Style</i> · ✓ What Is Math? 1–2 · ✓ How Computers Really Work (Dec)	CS61A final passed

Term	CS / CE blocks	Math	Science / Breadth	Reading this term	Milestone
Y1 Jan	4 · Nand2Tetris	—	—	—	Computer built from NAND
Y1 Spring	5 · SICP · 6 · C (✓ K&R, ✓ Zingaro I)	7 · Multivariable	8 · Physics II	Pinker, <i>Sense of Style</i> · ✓ What Is Math? 6–8 · ✓ Joy of X, ✓ Shape	Metacircular evaluator
Y2 Fall	9 · Computer Systems (✓ CS:APP)	10 · Math for CS (✓ Velleman 4–7) · 11 · Linear Algebra	HASS: Petzold, <i>Code</i>	Stanford <i>Writing in the Sciences</i> · ✓ What Is Math? 3–5	All 7 CS:APP labs
Y2 Jan	12 · Interpreters (✓ Ball → clox)	—	—	—	Bytecode VM with GC
Y2 Spring	13 · Algorithms I (✓ Zingaro II) · 14 · Architecture	15 · Probability	HASS: <i>Justice</i>	Zobel, <i>Writing for CS</i> · ✓ Algorithms to Live By	Pipelined RISC-V CPU
Y3 Fall	16 · OS (✓ How Linux Works, reread) · 17 · Software Construction	18 · Real Analysis	HASS: Econ 14.01	Booth, <i>Craft of Research</i>	All xv6 labs; 5k-word design doc
Y3 Jan	19 · Networking	—	—	—	TCP stack fetches a page

Term	CS / CE blocks	Math	Science / Breadth	Reading this term	Milestone
Y3 Spring	20 · Algorithms II · 21 · Databases	22 · Statistics	HASS: Chem or Bio	Schimmel, <i>Writing Science</i>	BusTub passes; 8k-word survey
Y4 Fall	23 · Distributed · 24 · Theory (✓ Hopcroft) · 26 · Spec A-1	25 · Convex Optimization	HASS: Ethics 6.805	Zinsser, <i>On Writing Well</i>	Raft passes 500×; capstone proposal
Y4 Jan	27 · Cryptopals or TLA+	—	—	—	—
Y4 Spring	28 · Spec A-2 · 29 · Spec B-1	—	HASS: <i>GEB</i>	Graff, <i>They Say / I Say</i>	Spec A complete
Y5 (MEng)	30 · Capstone · Spec B-2	Info theory (reading)	HASS ×2	McPhee, <i>Draft No. 4</i>	Thesis + talk + artifact

Always on: 500 words/day · Anki · daily log · nightly pleasure reading · from Y2: one Codeforces contest/month and two talks/year.

Part 1 — Phase 0: Prerequisites (0–5 months)

The first two blocks are new to most curricula and they're the reason the other 33 will actually get finished. They take about two weeks each. Do them first, in order, even if you can skip everything else in Phase 0.

P1 — Learning how to learn

- **Why:** Everything downstream assumes you know how memory, practice, and attention actually work. Most people don't, and they study in ways that feel productive and aren't. Two weeks here saves months later.

- **Do:** ✓ **Oakley, *A Mind for Numbers*** cover to cover, alongside her free Coursera course **Learning How to Learn** (Oakley & Sejnowski; ~15 hrs, four weeks of video, do it in two). Then **Brown, Roediger & McDaniel, *Make It Stick*** — the evidence behind Oakley: retrieval practice, spacing, interleaving, why rereading and highlighting feel effective and aren't. Finish with the paper that underlies both: **Dunlosky et al. (2013), "Improving Students' Learning With Effective Learning Techniques"** (free PDF) — read at least the summary table of which techniques work and which don't.
- **Also worth an evening:** **Scott Young, *Ultralearning*** — he did MIT's four-year CS curriculum in twelve months, and the book is his method. Skim it for the chapters on directness and drilling.
- **Build:** Your study system. Set up Anki and write your first twenty cards from these books. Write your weekly template (Appendix B.2). Write a one-page "how I will study" document in [/notes](#) — it becomes the thing you revise every six months.
- **Done when:** You can explain, without notes, retrieval practice, spacing, interleaving, focused vs. diffuse mode, and the illusion of competence — and your weekly template exists.
- **Hours:** ~30

P2 — Reading, thinking, and writing

- **Why:** You're about to read thousands of pages of textbooks and hundreds of papers. Reading a textbook, a proof, a paper, and a codebase are four different skills. So is solving a problem you've never seen. So is writing something a reader will finish.
- **Do — reading:** **Adler & Van Doren, *How to Read a Book***, Parts 1–3 (inspectional, analytical, and syntopical reading — the rest is optional). Then **Keshav, "How to Read a Paper"** (2007, 3 pages, free) for the three-pass method you'll use on every paper in this program. Then **Simonson & Gouvea, "How to Read Mathematics"** (free essay) — short and it changes how you sit with a proof. Then **Hermans, *The Programmer's Brain*** — cognitive science applied to reading and remembering code; nothing else covers this.
- **Do — thinking:** **Pólya, *How to Solve It***. Learn his four steps (understand, plan, carry out, look back) and use them, written out, on every hard problem for the first year until they're reflex.
- **Do — writing and speaking:** **Larry McEnerney, *The Craft of Writing Effectively*** (UChicago, YouTube, 80 min). Watch it twice. Then **Patrick Winston, *How to Speak*** (MIT OCW, 60 min). Start writing 500 words a day.
- **Also watch:** **Richard Hamming, "You and Your Research"** (1986, free) — the talk on how to do work that matters. Not about technique; about stance.
- **On speed reading:** Don't. Rayner et al. (2016, *PSPI*) reviewed the evidence; speed-reading gains come almost entirely from lost comprehension. Read at the speed of understanding and use retrieval practice instead: read a section, close the book, write what you remember, check.

- **Optional: Ahrens, *How to Take Smart Notes*** if you want a system for the [/notes](#) repo. Don't let note-taking become the hobby.
- **Build:** (1) An Adler-style analytical reading of one nonfiction book you've already read — write the one-page synopsis with its structure, claims, and where you disagree. (2) A three-pass read of one paper (Lampert's "Time, Clocks" or Ritchie & Thompson's "The UNIX Time-Sharing System" are good first ones). (3) Five hard math or puzzle problems solved with Pólya's four steps written out. (4) 14 straight days of 500 words.
- **Done when:** All four builds exist in [/notes](#).
- **Hours:** ~40

Skip any of the next three if you can pass the exit test. Don't skip on vibes.

P3 — Math prerequisites

- **Test:** Cold, on paper: simplify a nested log/exponent expression; solve a 3×3 linear system; expand $(a+b)^5$; prove $\sqrt{2}$ is irrational. Pass all four → skip Khan Academy, but still do Velleman.
- **Do:** Khan Academy Algebra II → Precalculus → Trigonometry as needed. Then ✓ **Velleman, *How to Prove It***, chapters 1–3, every exercise. Read ✓ **Lockhart, *Arithmetic*** alongside it in the evenings. If you've never done proof-based math, **Alcock, *How to Study as a Mathematics Major***, chapters 1–4, is a short, kind guide to what's about to change.
- **Done when:** The test passes and you can write a clean induction proof and a clean contradiction proof.
- **Hours:** 60–120

P4 — Programming on-ramp

- **Test:** Write a 300-line C program with manual memory management that's valgrind-clean; implement a hash table and BST from scratch. Can → skip.
- **Do:** **Harvard CS50x** (cs50.harvard.edu/x/). Every problem set, every lab, the final project.
- **Done when:** The test passes.
- **Hours:** 100–150

P5 — Tooling

- **Do:** MIT, ***The Missing Semester of Your CS Education*** (missing-semester.mit.edu/), all 11 lectures and exercises, with ✓ **Ward, *How Linux Works***, chapters 1–7, as the companion text. Set up permanently: native Linux (or WSL2 minimum), git + GitHub, vim or emacs, LaTeX via Overleaf, a [/notes](#) git repo with [log.md](#) (structure in Appendix B.5).
- **Done when:** [log.md](#) has 14 consecutive daily entries and you can navigate a Linux box without a GUI.

- **Hours:** ~40
-

Part 2 — The 30 blocks

Year 1 — Learn to program. Learn to prove.

Companion reading all year: ✓ *What Is Mathematics?* — one chapter every two weeks. Chapters 1–2 (numbers, number theory) in the fall; 6–8 (functions, limits, calculus) in the spring. It's the "why" behind the pssets.

Block 1 — Programming and Abstraction

- **Why:** Abstraction, recursion, higher-order functions, interpreters. The base layer of everything else.
- **Do: Berkeley CS61A** (cs61a.org). Every lecture, lab, homework, and all four projects, plus the free text *Composing Programs*.
- **Build:** The Scheme interpreter project. Then extend it with something the course doesn't ask for (tail calls, a macro system).
- **Done when:** A past CS61A final, timed and closed-book, at $\geq 70\%$.
- **Hours:** ~170

Block 2 — Calculus I

- **Do: MIT 18.01SC** (OCW Scholar) — lectures, recitations, all problem sets, both exams — with Strang's free *Calculus*.
- **Done when:** 18.01 final, timed, closed-book, passed.
- **Hours:** ~150

Block 3 — Physics I (breadth, lighter)

- **Why:** MIT requires it, and it's where the modeling instinct comes from.
- **Do: MIT 8.01SC**, ~7 hrs/wk, with ✓ **Feynman, *Six Easy Pieces*** as the companion — read the matching chapter before each unit.
- **Done when:** 8.01 final passed.
- **Hours:** ~100

Block 4 — Nand2Tetris (January intensive)

- **Why:** Build a whole computer, from NAND gates to a running program, in one month.
- **Prep (December):** Read the hardware half of ✓ **Justice, *How Computers Really Work*** — circuits, logic, and processor chapters. The gentle version of what you're about to build for real.

- **Do:** nand2tetris.org, both parts (free to audit on Coursera), with *The Elements of Computing Systems*, 2nd ed.
- **Build:** All twelve projects: logic gates → ALU → CPU → assembler → VM translator → Jack compiler → OS.
- **Done when:** A Jack program you wrote runs on the CPU you built.
- **Hours:** ~150

Block 5 — SICP

- **Why:** MIT's 6.001 for thirty years. CS61A's ideas at full depth, in Scheme.
- **Do:** Abelson & Sussman, *Structure and Interpretation of Computer Programs* (free, mitpress.mit.edu/sicp), chapters 1–4, with the 1986 MIT lectures on YouTube.
- **Build:** The metacircular evaluator (§4.1) and the lazy evaluator (§4.2).
- **Done when:** You can write the metacircular evaluator from memory, correctly, in under an hour.
- **Hours:** ~120

Block 6 — C fluency

- **Why:** Block 9 assumes real C. Get it now.
- **Do:** ✓ Kernighan & Ritchie, *The C Programming Language*, every exercise. Then ✓ Zingaro, *Algorithmic Thinking*, chapters 1–5 — C-based problem-solving, exactly the reps you need. Finish ✓ *How Linux Works* chapters 8–17 in the background.
- **Build:** A dynamic array, a string library, a hash table, and a small `ls` clone. Compile each with `-O2`, read the output of `objdump -d`, and annotate what the compiler did. (Assembly gets its full treatment in Block 9 — this is first contact.)
- **Done when:** Everything is valgrind-clean and you can explain pointer arithmetic, struct padding, and a stack frame on a whiteboard.
- **Hours:** ~110

Block 7 — Multivariable Calculus

- **Do:** MIT 18.02SC (Denis Auroux). All psets and exams.
- **Done when:** 18.02 final passed.
- **Hours:** ~150

Block 8 — Physics II (breadth, lighter)

- **Do:** MIT 8.02SC, ~7 hrs/wk.
- **Done when:** 8.02 final passed.
- **Hours:** ~100

Year 1 writing deliverable: 12 published blog posts explaining technical concepts, plus one 2,000-word essay with visible revision history. **Year 1 pleasure reading:** ✓ *The Joy of X*, ✓ *Shape*, and whatever fiction you want.

Year 2 — The systems year

Block 9 — Computer Systems

- **Why:** The single highest-payoff course in CS. Everything between your program and the hardware.
- **Do:** ✓ **Bryant & O'Hallaron, CS:APP, 3rd ed.**, chapters 1–12, with the **CMU 15-213** lecture videos.
- **Build:** All seven labs from csapp.cs.cmu.edu/3e/labs.html — Data, Bomb, Attack, Cache, Shell, Malloc, Proxy. Every one.
- **Done when:** All labs pass; malloc lab scores ≥ 90 on utilization and throughput; you read `objdump` output without flinching.
- **Hours:** ~200

Block 10 — Mathematics for Computer Science

- **Why:** Proofs, induction, counting, graphs, number theory, recurrences. The load-bearing math for theory and algorithms.
- **Do:** MIT **6.1200J / 6.042J** — the free textbook (Lehman, Leighton & Meyer) plus Tom Leighton's OCW lectures. Every problem set. Finish ✓ **Velleman** chapters 4–7 in the first month; read ✓ *What Is Mathematics?* chapters 3–5 alongside.
- **Done when:** 6.042 final passed, timed.
- **Hours:** ~170

Block 11 — Linear Algebra (taken twice, on purpose)

- **Why:** The most important math for CS. Once computationally, once abstractly.
- **Do:** MIT **18.06** (Strang, OCW) with all pssets. Then **Axler, Linear Algebra Done Right, 4th ed.** (free, open access), chapters 1–7.
- **Build:** From scratch in NumPy: LU with partial pivoting, modified Gram–Schmidt, QR, the power method. Compare against LAPACK and explain the differences.
- **Done when:** 18.06 final passed and Axler ch. 1–5 exercises done.
- **Hours:** ~200

Block 12 — Interpreters (January intensive)

- **Why:** Build two complete language implementations — one tree-walking, one bytecode.
- **Do:** ✓ **Ball, Writing an Interpreter in Go** — the whole book (the Monkey tree-walker). Then **Nystrom, Crafting Interpreters** (free), **Part III only** — the `clox` bytecode VM with a garbage collector, in C. Skip Nystrom's Part II; Ball covered it.
- **Build:** Monkey (Go), then `clox` (C). Bonus if days remain: Ball's *Writing a Compiler in Go*.
- **Done when:** `clox` passes the book's full test suite.

- **Hours:** ~130

Block 13 — Algorithms I

- **Do:** MIT 6.006 — the 2020 OCW version (Demaine, Ku, Solomon): lectures, recitations, all psets, exams. **CLRS 4th ed.** as the reference.
- **Practice:** ✓ **Zingaro** chapters 6–10 in parallel. Start **Codeforces**; one rated contest a month from here on. Evenings: ✓ *Algorithms to Live By*.
- **Done when:** 6.006 final passed, timed; Codeforces rating ≥ 1200 .
- **Hours:** ~180

Block 14 — Computer Architecture

- **Why:** How the CPU actually works. Also the start of the computer-engineering track.
- **Do:** Onur Mutlu, *Digital Design and Computer Architecture*, ETH Zürich (Spring 2025 — full lectures, slides, labs, and exams at safari.ethz.ch and youtube.com/onurmutlulectures), with Harris & Harris, *Digital Design and Computer Architecture, RISC-V Edition*.
- **Build:** A 5-stage pipelined RISC-V core in Verilog with forwarding and hazard detection. Simulate with Verilator; then put it on an FPGA (Digilent Arty A7 or iCEBreaker).
- **Done when:** Your CPU runs a C program you compiled with `riscv64-gcc`.
- **Hours:** ~200

Block 15 — Probability

- **Do:** MIT 6.3700 / 6.041 (John Tsitsiklis, OCW) with the Bertsekas & Tsitsiklis text. All psets.
- **Done when:** Final passed; you can derive the standard distributions and their moments, and solve Markov-chain problems without a formula sheet.
- **Hours:** ~150

Year 2 writing deliverable: 12 posts; a 5,000-word technical design document for your CPU or interpreter; one paper summary per month in the three-pass format (P2).

Year 3 — Depth

Block 16 — Operating Systems

- **Prep:** Reread ✓ **Ward, How Linux Works** — all of it, fast. The user-space view of what you're about to build from the kernel side.
- **Do:** Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (free) alongside MIT 6.1810 (pdos.csail.mit.edu/6.1810; also OCW Fall 2023). xv6 on RISC-V.

- **Build:** Every xv6 lab — util, syscall, page tables, traps, copy-on-write, networking, locks, file system, mmap — graded with **make grade**.
- **Done when:** All labs pass. Then: a minimal kernel of your own that boots on QEMU and schedules two processes.
- **Hours:** ~200

Block 17 — Software Construction

- **Why:** The most under-appreciated course in the MIT degree. How to write code that doesn't rot.
- **Do:** MIT 6.1020 / 6.031 readings and problem sets (web.mit.edu/6.031/www) plus **Ousterhout, *A Philosophy of Software Design*, 2nd ed.**
- **Build:** Take your **c1ox** or Monkey interpreter and rebuild it under test: specifications first, rep invariants, abstraction functions, then a real test strategy.
- **Done when:** You write a rep invariant and abstraction function for any type you define, by reflex.
- **Hours:** ~150

Block 18 — Real Analysis

- **Why:** Where you learn what a proof actually is. The hardest block for most people and the one that changes your thinking the most.
- **Do:** **Abbott, *Understanding Analysis*** — the whole book — then **MIT 18.100A** (OCW) for problems and exams. If you stall, **Alcock, *How to Think About Analysis*** is a short book about exactly this stall.
- **Done when:** You can prove Bolzano–Weierstrass, the extreme value theorem, and a uniform-continuity result from the definitions, on paper, unaided.
- **Hours:** ~180

Block 19 — Networking (January intensive)

- **Do:** **Stanford CS144** (cs144.github.io) with **Kurose & Ross, *Computer Networking: A Top-Down Approach***.
- **Build:** All eight labs — a working TCP implementation in C++.
- **Done when:** Your TCP stack fetches a real web page over the real internet.
- **Hours:** ~150

Block 20 — Algorithms II

- **Do:** MIT 6.046J (OCW) with **Kleinberg & Tardos, *Algorithm Design***. Divide-and-conquer, DP, greedy, flow, LP, randomized, NP-completeness, approximation.
- **Done when:** 6.046 final passed; Codeforces ≥ 1600 ; you can solve an unseen medium-hard problem in 45 minutes with a correctness argument and tight bound.
- **Hours:** ~180

Block 21 — Databases

- **Do:** **CMU 15-445/645** (Andy Pavlo; 15445.courses.cs.cmu.edu) — the latest term's lectures and notes. Non-CMU students can submit to Gradescope with the public code posted each term.
- **Build:** All four **BusTub** projects in C++: buffer pool manager, B+ tree, query executors and optimizer, concurrency control. Then read **Kleppmann, *Designing Data-Intensive Applications*** cover to cover.
- **Done when:** All four projects pass Gradescope.
- **Hours:** ~200

Block 22 — Statistics

- **Do:** **Wasserman, *All of Statistics***, chapters 1–13, plus **McElreath's *Statistical Rethinking* lectures** (free, YouTube).
- **Done when:** On a real dataset, you can do MLE, confidence intervals, hypothesis tests, and Bayesian inference with MCMC, and explain when each is the wrong tool.
- **Hours:** ~150

Year 3 writing deliverable: An 8,000-word survey of one subfield with 30+ cited sources; two public talks, recorded.

Year 4 — Advanced core and Specialization A

Block 23 — Distributed Systems

- **Do:** **MIT 6.5840** (formerly 6.824; pdos.csail.mit.edu/6.824) — Robert Morris's lectures on YouTube, every assigned paper, plus **Martin Kleppmann's Cambridge distributed systems lectures** (free) for theory.
- **Build:** Labs 1–5 in Go: MapReduce, Raft, fault-tolerant KV store, sharded KV store.
- **Done when:** All labs pass 500 consecutive runs under `go test -race`.
- **Hours:** ~220

Block 24 — Theory of Computation

- **Do:** ✓ **Hopcroft, Motwani & Ullman, *Introduction to Automata Theory, Languages, and Computation***, with **MIT 6.1400 / 6.045** (OCW) for lectures, psets, and exams. HMU is denser than Sipser; if a chapter isn't landing, read Sipser's version and come back.
- **Done when:** 6.045 final passed; you can prove a problem you've never seen is NP-complete, and prove undecidability by reduction.
- **Hours:** ~170

Block 25 — Convex Optimization

- **Do:** **Boyd & Vandenberghe, *Convex Optimization*** (free PDF) with **Stanford EE364A** videos and homework.
- **Build:** Formulate and solve a real problem in CVXPY (portfolio, control, or an ML objective). Derive its KKT conditions by hand.
- **Done when:** EE364A homework sets 1–8 done.
- **Hours:** ~150

Block 26 — Specialization A, course 1

- See Part 3. Pick your track and do its first course.

Block 27 — January intensive: pick one

- **Option A — Cryptopals** (cryptopals.com): all eight sets. The best hands-on cryptography education anywhere.
- **Option B — TLA+:** Lamport's free TLA+ Video Course, then **Hillel Wayne, *Practical TLA+***. Write a spec of your Raft from Block 23 and find a real bug in it.
- **Hours:** ~130

Block 28 — Specialization A, course 2

Block 29 — Specialization B, course 1

Year 4 writing deliverable: Capstone proposal (3,000 words: problem, related work, plan, evaluation criteria). Continue posts and talks.

Year 5 — The MEng year

Block 30 — Capstone

Two semesters. This is what makes the program MEng-equivalent rather than SB-equivalent. Pick one:

- **A. Build a serious system** — a database, a language with a compiler, a kernel, a distributed store, a renderer, an ML framework. 10,000+ lines, real tests, benchmarks against an existing system.
- **B. Replicate and extend research** — one paper from the last three years of OSDI, SOSP, SIGCOMM, PLDI, ISCA, NeurIPS, or SIGMOD. Reproduce its results. Then extend it.
- **C. Substantial open-source contribution** — ship a real feature in Linux, LLVM, PostgreSQL, Rust, CPython, SQLite, or Kubernetes. Get it reviewed and merged.

- **D. Tape out a chip** — via Tiny Tapeout or Efabless with the open-source SkyWater flow.

Deliverables regardless of option: the public artifact with documentation · a 15,000–25,000-word thesis (motivation, related work, design, implementation, evaluation, limitations) · a 30-minute recorded talk · written feedback from at least one outside reviewer who read all of it. Consider submitting to a workshop.

Alongside: Specialization B course 2, and **MacKay, *Information Theory, Inference, and Learning Algorithms*** (free) as ~4 hrs/wk reading.

Full-time note: Years 4 and 5 compress into about 18 months if you have the hours.

Part 3 — Specializations (pick two)

Each track is two courses plus a build. Do Track A's two courses in Year 4 (Blocks 26, 28) and Track B's in Years 4–5 (Block 29 + alongside capstone). **Two deep beats six shallow.**

Track 1 — AI and Machine Learning

- **Course 1: Stanford CS229** (Andrew Ng — the real course, not the Coursera intro) with **Bishop, *Deep Learning: Foundations and Concepts*** (free online at bishopbook.com).
- **Course 2: Karpathy, *Neural Networks: Zero to Hero*** (free) — backprop → makemore → GPT from scratch — then **Stanford CS231n** or **CS224n** depending on interest.
- **Build:** A transformer, from scratch, that trains to a reasonable loss. Then **CMU 10-414 *Deep Learning Systems*** (dlsyscourse.org): build a PyTorch-like framework.
- **Extend:** Sutton & Barto + Berkeley CS285 for RL; MIT 6.5940 (Song Han) for efficient ML.

Track 2 — Systems and Performance

- **Course 1: MIT 6.172 *Performance Engineering*** (OCW, Leiserson) — cache-aware algorithms, vectorization, parallelism, profiling.
- **Course 2: CMU 15-721 *Advanced Database Systems*** (Pavlo) or **Herlihy & Shavit, *The Art of Multiprocessor Programming*** worked as a course.
- **Build:** Merged patches to Linux, PostgreSQL, LLVM, or a language runtime. The most legible credential in systems.
- **Books:** Gregg, *Systems Performance* 2e; Agner Fog's optimization manuals (free).

Track 3 — Security and Cryptography

- **Course 1: Dan Boneh, *Cryptography I*** (Coursera, free) with **Boneh & Shoup, *A Graduate Course in Applied Cryptography*** (free).

- **Course 2: MIT 6.1600 / 6.858 Computer Systems Security** (OCW, public labs) plus **pwn.college** (ASU, free — binary exploitation, reversing, kernel).
- **Build:** All of Cryptopals (if not done in Block 27); a real CTF placement.
- **Books:** Anderson, *Security Engineering* 3e (free); Katz & Lindell for rigor.

Track 4 — Graphics and Vision

- **Course 1: GAMES101** (Lingqi Yan; free, English lectures, excellent assignments) — the best intro graphics course available.
- **Course 2: CMU 15-462 or Berkeley CS184**, with **Pharr, Jakob & Humphreys**, *Physically Based Rendering*, 4th ed. (free, pbr-book.org).
- **Build:** A path tracer with importance sampling and a BVH; then a real-time renderer in Vulkan or WebGPU.
- **Vision:** Stanford CS231n; Szeliski, *Computer Vision* 2e (free); Hartley & Zisserman.

Track 5 — Programming Languages and Compilers

- **Course 1: Dan Grossman, Programming Languages** (UW, Coursera, free to audit — SML, Racket, Ruby) with **Pierce, Types and Programming Languages**.
- **Course 2: Cornell CS 6120 Advanced Compilers** (Adrian Sampson — built to be self-guided) with **Cooper & Torczon, Engineering a Compiler**, 3rd ed.
- **Build:** A compiler for a small language targeting real x86-64 or RISC-V with register allocation and three optimization passes. Ball's *Writing a Compiler in Go* is a good warm-up if you didn't do it in Block 12. Then **Pierce et al., Software Foundations** Vol. 1–2 (free, Coq/Rocq).
- **Extend:** OPLSS lectures (free); Chlipala's *Certified Programming with Dependent Types* (free).

Track 6 — Computer Engineering (deep hardware)

- **Course 1: Onur Mutlu, Graduate Computer Architecture**, ETH Zürich (free; out-of-order, memory systems, coherence) with **Hennessy & Patterson, Computer Architecture: A Quantitative Approach**, 6th ed.
- **Course 2: MIT 6.003 / 6.3000 Signals and Systems** (OCW, Oppenheim) — required for anything touching DSP, comms, or control.
- **Build:** Extend your Block 14 core with branch prediction, a cache hierarchy, and out-of-order execution. Characterize it in gem5. Then tape out via Tiny Tapeout.
- **Extend:** MIT 6.002 circuits (Agarwal & Lang) + Horowitz & Hill; Lee & Seshia, *Introduction to Embedded Systems* (free) with bare-metal STM32/RP2040 firmware; Weste & Harris, *CMOS VLSI Design* with the OpenLane/SkyWater flow. ✓ *How Computers Really Work* has a hands-on Raspberry Pi/circuits half to revisit here.

If you want the full computer-engineering degree rather than a specialization:
add 6.002 Circuits in Y2 Spring, 6.003 Signals in Y3 Spring, and embedded

firmware as the Y3 January intensive alongside CS144. That gets you to roughly MIT 6-5 hardware competence.

Part 4 — The five always-on habits

These never pause. They're the difference between finishing and not. P1 explains why each works.

Habit 1 — Write 500 words a day

Five days a week, for the whole program. One writing book per term (the "Reading" column in the map). Yearly deliverables are at the end of each year in Part 2. Publish the posts. Full writing list in Appendix C.

Habit 2 — Anki, daily

Cards for things that are pure recall and decay: complexity classes, syscall semantics, cache-coherence states, distributions and their moments, TCP state machine, LaTeX. **Not** for skills. Write your own cards, one fact each, only after you understand the thing.

Habit 3 — One breadth subject per term

MIT requires eight humanities subjects and six science subjects. The map assigns one per term, ~4 hrs/wk. Details in Appendix C. Plus: one foreign language, to B1, across the whole program — half an hour a day.

Habit 4 — The daily log

[log.md](#) in git. What you did, what you got stuck on, what you'll do tomorrow. Two minutes. Fifteen years from now this is the most valuable file you own.

Habit 5 — Read for pleasure, nightly

Twenty to thirty minutes before bed, anything you actually want to read. Your Weir novels are ideal — problem-solving as story. Fantasy, thrillers, whatever: it all counts. The literary list in Appendix C is for when you want to deliberately absorb better prose, not a replacement for what you enjoy. Volume matters more than selection.

From Year 2: one Codeforces contest per month and two recorded talks per year.

Part 5 — Schedules

5.1 Full-time (~45 hrs/wk)

This is the map at the top of the document. Four blocks per term plus habits. Years 1–4 give you the SB-equivalent with one specialization; Year 5 adds the second specialization and the capstone. Full-timers can compress Years 4–5 into ~18 months.

5.2 Part-time (~20 hrs/wk)

Two blocks per term, one of them math or breadth when possible. Habits continue at reduced volume (300 words/day is fine). Same order.

Year	Fall	Jan	Spring
1	Phase 0 → Block 1	—	Block 2 · Block 3
2	Block 4 · Block 5	—	Block 6 · Block 7
3	Block 9 · Block 8	Block 12 (vacation week + weekends)	Block 10 · Block 11
4	Block 13 · Block 15	—	Block 14 · Block 17
5	Block 16 · Block 18	Block 19 (vacation week + weekends)	Block 20 · Block 22
6	Block 21 · Block 24	—	Block 23 · Block 25
7	Block 26 · Block 27	—	Block 28 · Block 29
8	Block 30 (capstone)	—	Block 30 (capstone)

Seven and a half years at 20 hrs/wk. If that's too long, cut scope (skip Physics, skip Block 22, do one specialization) — don't stretch the timeline further; past ~8 years the completion rate goes to zero.

Part-time rules:

- Never run two systems blocks (9, 14, 16, 19, 21, 23) at the same time.
- Take one week of vacation per year for the January intensive.
- If you fall two weeks behind, don't catch up — drop the lighter block for the term.

5.3 The 18-month "get employable, backfill later" version

If you need a job fast, do exactly these and nothing else, with 500 words/day throughout:

P1 → P2 → Block 1 → Block 6 → Block 9 → Block 10 → Block 13 → Block 16 → Block 19 → Block 21 → DDIA → one serious project.

This is not the program. Be honest with yourself about which one you're doing.

Appendix A — Alternatives and second sources

Only open this if the primary resource in a block genuinely isn't working. Organized by block.

Block	Alternatives
P1 Learning	Newport, <i>Deep Work</i> (focus); Ericsson & Pool, <i>Peak</i> (deliberate practice); Newport, <i>How to Become a Straight-A Student</i> (tactics); Matuschak, "Why Books Don't Work" (free essay — provocative, worth an hour)
P2 Reading/thinking	Ahrens, <i>How to Take Smart Notes</i> ; Zinsser's <i>Writing to Learn</i> ; Alcock, <i>How to Study as a Mathematics Major</i> (full book)
P3 Proofs	Hammack, <i>Book of Proof</i> (free) if Velleman's structured style isn't clicking; Lockhart's essay <i>A Mathematician's Lament</i> (free)
1 Programming	MIT 6.1010 (OCW); <i>How to Design Programs</i> (Felleisen, free); Exercism for reps in any language
2, 7 Calculus	Spivak, <i>Calculus</i> (proof-based — better long-term if you have time); Apostol; Professor Leonard (YouTube)
3, 8 Physics	Kleppner & Kolenkow (mechanics), Purcell (E&M) for rigor; <i>The Feynman Lectures</i> (free — <i>Six Easy Pieces</i> is excerpted from them)

Block	Alternatives
4 Nand2Tetris	No real substitute. Do it.
5 SICP	<i>SICP: JavaScript Edition</i> ; Brian Harvey's CS61A Scheme lectures (YouTube)
6 C	<i>Modern C</i> (Gustedt, free); CS50's C weeks; <i>Effective C</i> (Seacord); <i>Programming from the Ground Up</i> (free) for a gentle assembly intro
9 Systems	Berkeley CS61C (cs61c.org) — RISC-V-based, full labs; <i>Dive Into Systems</i> (free)
10 Discrete	Rosen as reference; Graham, Knuth & Patashnik, <i>Concrete Mathematics</i> for the hard version
11 Linear algebra	3Blue1Brown <i>Essence of Linear Algebra</i> (intuition only); Trefethen & Bau, <i>Numerical Linear Algebra</i> (the applied follow-up — do it in Year 3 if you can)
12 Interpreters	Crafting Interpreters Part II (jlox) if you want Nystrom's tree-walker too; Ball, <i>Writing a Compiler in Go</i>
13, 20 Algorithms	Erickson, <i>Algorithms</i> (free, best problem sets in the genre); Skiena, <i>Algorithm Design Manual</i> 3e + his lectures; Roughgarden's <i>Algorithms Illuminated</i> + Stanford CS161; Princeton Algorithms (Sedgewick, Coursera); MIT 6.851 and 6.854 for graduate depth; Laaksonen's <i>Competitive Programmer's Handbook</i> (free)
14 Architecture	MIT 6.1910 / 6.004 (OCW); Berkeley CS61C's project 3; Patterson & Hennessy, <i>Computer Organization and Design</i> , RISC-V ed.; Chisel instead of Verilog

Block	Alternatives
15 Probability	Harvard Stat 110 (Blitzstein — free lectures, free book, free psets with solutions). Equally good; pick the voice you prefer.
16 OS	Berkeley CS162 (Pintos); Nanjing University OS (jyy — excellent, English materials); Tanenbaum, <i>Modern Operating Systems</i> ; Love, <i>Linux Kernel Development</i>
17 Software construction	<i>Refactoring 2e</i> (Fowler); <i>Working Effectively with Legacy Code</i> (Feathers); <i>The Pragmatic Programmer</i> 20th anniv.; <i>Software Engineering at Google</i> (free); <i>Site Reliability Engineering</i> (free)
18 Analysis	Rudin, <i>Principles of Mathematical Analysis</i> (terse, the standard); Tao, <i>Analysis I & II</i> (builds from $\mathbb{N}$ — great for self-study)
19 Networking	Berkeley CS168; Peterson & Davie, <i>Computer Networks: A Systems Approach</i> (free); Stevens, <i>TCP/IP Illustrated</i> Vol. 1 as permanent reference; Beej's Guide
21 Databases	Berkeley CS186 (Java); Petrov, <i>Database Internals</i> ; the Red Book (redbook.io); Silberschatz
22 Statistics	MIT 6.3800 <i>Introduction to Inference</i> ; Gelman et al., <i>Regression and Other Stories</i>
23 Distributed	van Steen & Tanenbaum, <i>Distributed Systems</i> (free); the core papers: Lamport "Time, Clocks", Paxos Made Simple, Raft, MapReduce, GFS, Bigtable, Dynamo, Spanner, FLP, CAP
24 Theory	Sipser , <i>Introduction to the Theory of Computation</i> , 3e (the gentler standard — buy it if HMU fights you); Barak , <i>Introduction to Theoretical Computer</i>

Block	Alternatives
	Science (free, introcs.org); Arora & Barak for complexity
25 Optimization	EE364B (Stanford) for the sequel; Nocedal & Wright for non-convex

Extra math if your specialization needs it: abstract algebra (Artin; Harvard Math 122 videos free) for crypto/coding theory; Durrett (free) for measure-theoretic probability; Mitzenmacher & Upfal for randomized algorithms; Spielman's spectral graph theory notes (free); Milewski's *Category Theory for Programmers* (free) for PL; Enderton for logic. **Evan Chen's Napkin** (free) is the best map of the whole territory — ✓ *What Is Mathematics?* is the classic version of the same idea.

Paid platforms worth the money: **CS Primer** (csprimer.com — Oz Nova, author of teachyourselfcs.com; self-paced, problem-driven, architecture through distributed systems); **Codecrafters** (build-your-own Redis/Git/SQLite/shell/HTTP server; ~\$30–40/mo); **Bradfield CS Intensive** (live, small-cohort, expensive, excellent).

Curated indexes: teachyourselfcs.com (tightest curation, nine subjects); csdiy.wiki/en (huge, maintained index of the best global courses); [ossu/computer-science](https://ossu.computer-science) on GitHub (free curriculum with an active Discord).

Appendix B — The study rules, in detail

P1 gives you the science. This is the CS-specific application.

B.1 How to grade yourself without a TA

Ranked by usefulness:

1. **Autograders.** [make grade](#) in 6.1810. Gradescope for 15-445. Test suites in CS144, BusTub, clox, Monkey, 6.5840. Use them.
2. **Past exams, timed, closed-book.** MIT OCW, Berkeley's exam archive (hkn.eecs.berkeley.edu/exams), CMU course sites. Sit them under real conditions. This is your only objective signal.
3. **Write the solution as if for publication.** If the write-up is vague, so is your understanding.
4. **Code review from strangers.** Open a PR against a real project.
5. **Teach it.** A blog post explaining the hardest idea in the block.

B.2 The weekly shape (part-time)

- Weekday mornings, 90 min before work: the hardest material (proofs, theory, algorithms). Protected.
- Weekday evenings, 60 min: lectures, reading, Anki.
- Saturday, 4–6 hrs: the build block. Systems projects live here.
- Sunday, 2 hrs: psets, writing, weekly review, plan next week. About 20 hrs/wk and sustainable for years, which is the only property that matters. Write your own version of this in P1.

B.3 Time, honestly

MIT counts one "unit" as roughly one hour per week for a 14-week term; a 12-unit subject is ~170 hours. The whole program is roughly **6,500–8,000 hours**. Below 15 hrs/wk, cut scope rather than extending the timeline.

B.4 Community — do not skip

- **Recurse Center** (recurse.com): free, self-directed 6- or 12-week retreat, remote or NYC. The highest-value single thing available to a self-taught programmer. Apply after Block 12 or so.
- **A study partner**. One person on the same path, weekly video call, screen-share psets. Roughly doubles completion rates.
- **Papers We Love**, the OSSU Discord, language Discords (Rust, Zig, Haskell), local university lectures — many let you audit free.

B.5 Notes

/notes

how-i-study.md # written in P1, revised every 6 months

log.md # daily

/math /systems /theory ... # one file per topic

/papers # one file per paper, three-pass format

/writing # every essay, dated

/projects # one repo per build

Appendix C — Writing and breadth reading lists

C.1 Writing books, in the order the map assigns them

1. **Williams & Bizup, *Style: Lessons in Clarity and Grace*** — the single most useful book on English prose. Do the exercises. Reread every two years.
2. **Pinker, *The Sense of Style*** — read chapter 3 ("The Curse of Knowledge") twice.
3. **Stanford, *Writing in the Sciences*** (Kristin Sainani; free, 8 weeks, Coursera/Stanford Online) — clutter, verbs, paragraphs, manuscript structure, peer review.
4. **Zobel, *Writing for Computer Science*, 3rd ed.** — algorithms, experiments, honest graphs.
5. **Booth, Colomb & Williams, *The Craft of Research*, 5th ed.** — the closest thing to a research-methods course in book form.
6. **Schimmel, *Writing Science*** — story structure for technical papers. Pair with **Knuth, Larrabee & Roberts, *Mathematical Writing*** (free PDF).
7. **Zinsser, *On Writing Well*** — voice and ruthlessness about clutter.
8. **Graff & Birkenstein, *They Say / I Say*** — templates for entering an academic conversation. Feels remedial; isn't.
9. **McPhee, *Draft No. 4*** — the craft, from someone who has it.

Also worth owning: Thomas & Turner, *Clear and Simple as the Truth*; Dupré, *BUGS in Writing* (CS-specific errors); Heinrichs, *Thank You for Arguing*; Prose, *Reading Like a Writer*; the Google developer documentation style guide (free). (Adler's *How to Read a Book* is in P2.)

If English isn't your first language: Swan, *Practical English Usage*, 4th ed. as the reference; Huddleston & Pullum, *A Student's Introduction to English Grammar* if you want the system.

When you want to absorb better prose on purpose (in addition to, not instead of, what you enjoy): Orwell's essays (start with "Politics and the English Language"), Didion's *Slouching Towards Bethlehem*, Feynman's *Surely You're Joking* (you already like Feynman), Chiang's *Stories of Your Life* (if you like Weir, you'll like this), Borges' *Ficciones*, Levi's *The Periodic Table*. And the best CS prose: Dijkstra's EWDs, Brooks' *Mythical Man-Month*, Lampson's "Hints for Computer System Design," Lamport's "Time, Clocks."

Speaking: Winston's lecture (P2), then Toastmasters or a local meetup. Two talks a year, recorded, watched back.

C.2 Breadth subjects, in the order the map assigns them

Term	Subject	Do
Y2 Fall	History of computing	Petzold, <i>Code</i> , 2nd ed.; then <i>The Annotated Turing</i>
Y2 Spring	Philosophy	Sandel's Harvard <i>Justice</i> (free, justiceharvard.org); Nagel, <i>What Does It All Mean?</i>
Y3 Fall	Economics	MIT 14.01 Microeconomics (OCW)
Y3 Spring	Chemistry or biology	MIT 5.111 or 7.01SC (OCW) — the sixth MIT science subject
Y4 Fall	Ethics of technology	MIT 6.805 (OCW); O'Neil, <i>Weapons of Math Destruction</i>
Y4 Spring	Cognitive science	Hofstadter, <i>Gödel, Escher, Bach</i> — slowly
Y5	History	One Yale Open Course
Y5	Arts or literature	A great-books sequence, or Prose's <i>Reading Like a Writer</i> applied to three novels

Plus one foreign language to B1 across the whole program.

Appendix D — Credentials, budget, and what to buy next

D.1 Credentials that actually change outcomes

Option	Cost	Notes
Recurse Center	free	Apply. Nothing else has this value-per-dollar.

Option	Cost	Notes
Georgia Tech OMSCS	~\$7–8k total	Accredited MS, rigorous, online. Needs a bachelor's in <i>any</i> field.
WGU BS Computer Science	~\$4–5k per 6-month term	Accredited, competency-based. If you have no bachelor's, this curriculum will let you finish it fast.
University of London BSc CS (Coursera)	~\$5–6k/yr	Accredited UK degree, fully online.
UT Austin MSCSO / UIUC MCS	~\$10–15k	Same idea as OMSCS.
CS Primer / Bradfield / Codecrafters	varies	See Appendix A.

D.2 Budget

- **Minimum:** ~\$0 beyond what you own. Everything essential is free online or at a library.
- **Recommended (~\$1,200–2,000 over the program):** ~20 more physical books, mostly used (\$500–800); FPGA board + logic analyzer (\$250–400); STM32/RP2040 kit, breadboard, scope, multimeter (\$200–300); one paid platform for a year (\$400–700); cloud credits (\$200–400).
- **If money is available (~\$12–20k):** add OMSCS, a Tiny Tapeout run (\$300–500), and one conference a year (USENIX ATC, SOSOP, or regional — the hallway is the point).
- **Buy used.** Older editions of CLRS, Kurose & Ross, and Patterson & Hennessy are 95% identical at 10% of the price.

D.3 What to buy next, in the order you'll need it

You already own K&R, CS:APP, Zingaro, Ball, Hopcroft, Velleman, and *A Mind for Numbers* — seven of the program's primary texts. Next:

Phase 0 (all cheap used, or any library):

1. **Brown, Roediger & McDaniel, *Make It Stick*** — P1
2. **Adler & Van Doren, *How to Read a Book*** — P2
3. **Pólya, *How to Solve It*** — P2
4. **Hermans, *The Programmer's Brain*** — P2

Year 1: 5. Williams & Bizup, *Style* — Y1 Fall 6. Nisan & Schocken, *The Elements of Computing Systems*, 2e — Y1 Jan (site is free; the book is worth having) 7. Pinker, *The Sense of Style* — Y1 Spring

Year 2 onward: 8. CLRS, *Introduction to Algorithms*, 4e — Y2 Spring (used is fine) 9. Harris & Harris, *Digital Design and Computer Architecture*, RISC-V ed. — Y2 Spring 10. Ousterhout, *A Philosophy of Software Design*, 2e — Y3 Fall 11. Abbott, *Understanding Analysis* — Y3 Fall 12. Kleinberg & Tardos, *Algorithm Design* — Y3 Spring 13. Kleppmann, *Designing Data-Intensive Applications* — Y3 Spring 14. Sipser, *Theory of Computation*, 3e — Y4 Fall, only if HMU is fighting you

Everything else is free online or can wait until its block.

Appendix E — Failure modes, in order of frequency

1. **Course collecting.** Twelve things enrolled, none finished. *Fix: Rule 3.*
 2. **Watching lectures instead of doing problems.** The most seductive failure — P1 explains why it feels like learning. *Fix: Rule 1.*
 3. **Skipping math**, hitting a wall in Year 3, quitting. *Fix: Rule 4.*
 4. **Skipping writing**, then stalling at the design-doc stage at work. *Fix: Habit 1.*
 5. **Tutorial-difficulty projects.** *Fix: every build must contain a part where you're stuck for a day.*
 6. **No feedback loop.** *Fix: publish, contribute, study partner, Recurse.*
 7. **LeetCode as curriculum.** It teaches pattern-matching, not design. *Fix: six-week sprint before job hunting, not before.*
 8. **Comparing to other people's highlight reels.** *Fix: compare to your own log from six months ago.*
 9. **Building on sand.** If you can't explain cache lines or process-vs-thread cold, you're not ready for distributed systems. *Fix: the "Done when" lines, honestly applied.*
 10. **A 50-hour week you can't sustain.** *Fix: pick the honest pace and hold it for years.*
 11. **Skipping P1 and P2 because they don't feel like "real" CS.** They're the two blocks that determine whether the other 33 get done.
-

Appendix F — URLs

Resource	URL
MIT OpenCourseWare	ocw.mit.edu
MIT course renumbering (old ↔ new)	eecs.mit.edu/numbering.html
MIT 6-3 degree chart	catalog.mit.edu/degree-charts/computer-science-engineering-course-6-3/
Learning How to Learn (Oakley)	coursera.org/learn/learning-how-to-learn
Keshav, "How to Read a Paper"	search the title — free PDF, 3 pages
Hamming, "You and Your Research"	search the title — transcript and video both free
Missing Semester	missing-semester.mit.edu
CS50x	cs50.harvard.edu/x
CS61A	cs61a.org
Nand2Tetris	nand2tetris.org
SICP	mitpress.mit.edu/sicp
CS:APP labs	csapp.cs.cmu.edu/3e/labs.html
Math for CS text	ocw.mit.edu → 6.042J
Axler LADR 4e	linear.axler.net
Crafting Interpreters	craftinginterpreters.com
ETH DDCA (Mutlu)	safari.ethz.ch · youtube.com/onurmutlulectures
OSTEP	pages.cs.wisc.edu/~remzi/OSTEP
MIT 6.1810	pdos.csail.mit.edu/6.1810
MIT 6.1020 / 6.031	web.mit.edu/6.031/www
Stanford CS144	cs144.github.io

Resource	URL
CMU 15-445	15445.courses.cs.cmu.edu
MIT 6.5840	pdos.csail.mit.edu/6.824
Boyd, Convex Optimization	stanford.edu/~boyd/cvxbook · stanford.edu/class/ee364a
Barak, Intro to TCS	introtcs.org
Cryptopals	cryptopals.com
pwn.college	pwn.college
Cornell CS 6120	cs.cornell.edu/courses/cs6120
CMU 10-414 DL Systems	dlsyscourse.org
PBRT 4e	pbr-book.org
Stanford Writing in the Sciences	online.stanford.edu → "Writing in the Sciences"
Recurse Center	recurse.com
Teach Yourself CS	teachyourselfcs.com
csdiy.wiki (English)	csdiy.wiki/en
CS Primer	csprimer.com

Appendix G — The checklist

Print it. Check boxes only when the *Done when* line is true.

Phase 0

- ☐ P1 Learning how to learn (✓ Mind for Numbers, *Learning How to Learn*, *Make It Stick*, Dunlosky) — study system written
- ☐ P2 Reading, thinking, writing (Adler, Keshav, Pólya, Hermans, McEnerney, Winston) — four builds in [/notes](#)
- ☐ P3 Math prerequisites (✓ Velleman 1–3, ✓ Arithmetic)

- ☐ P4 Programming on-ramp
- ☐ P5 Tooling + notes repo (✓ How Linux Works 1–7)

Year 1

- ☐ 1 CS61A
- ☐ 2 Calculus I
- ☐ 3 Physics I (✓ Six Easy Pieces)
- ☐ ✓ *How Computers Really Work* — hardware half (Dec)
- ☐ 4 Nand2Tetris
- ☐ 5 SICP
- ☐ 6 C fluency (✓ K&R, ✓ Zingaro 1–5, ✓ How Linux Works 8–17)
- ☐ 7 Multivariable
- ☐ 8 Physics II
- ☐ ✓ *What Is Mathematics?* ch. 1–2, 6–8
- ☐ Y1 writing: 12 posts + 2,000-word essay

Year 2

- ☐ 9 Computer Systems (✓ CS:APP) — all 7 labs
- ☐ 10 Math for CS (✓ Velleman 4–7, ✓ What Is Math? 3–5)
- ☐ 11 Linear Algebra
- ☐ 12 Interpreters (✓ Ball → clox)
- ☐ 13 Algorithms I (✓ Zingaro 6–10, ✓ Algorithms to Live By)
- ☐ 14 Computer Architecture
- ☐ 15 Probability
- ☐ Y2 writing: 12 posts + 5,000-word design doc + monthly paper summaries

Year 3

- ☐ 16 Operating Systems (✓ How Linux Works reread)
- ☐ 17 Software Construction
- ☐ 18 Real Analysis
- ☐ 19 Networking
- ☐ 20 Algorithms II
- ☐ 21 Databases
- ☐ 22 Statistics
- ☐ Y3 writing: 8,000-word survey + 2 talks

Year 4

- ☐ 23 Distributed Systems
- ☐ 24 Theory of Computation (✓ Hopcroft)
- ☐ 25 Convex Optimization
- ☐ 26 Specialization A — course 1
- ☐ 27 January intensive (Cryptopals or TLA+)
- ☐ 28 Specialization A — course 2
- ☐ 29 Specialization B — course 1
- ☐ Y4 writing: capstone proposal

Year 5

- ☐ 30 Capstone: artifact
- ☐ 30 Capstone: thesis
- ☐ 30 Capstone: talk
- ☐ 30 Capstone: outside review
- ☐ Specialization B — course 2
- ☐ Information theory (MacKay)

Habits — running tally

- ☐ 500 words/day streak: ____ days
- ☐ Anki: daily since ____
- ☐ Breadth subjects: ____ / 8
- ☐ Codeforces rating: ____
- ☐ Talks given: ____
- ☐ Foreign language level: ____
- ☐ Books read for pleasure this year: ____
- ☐ [how-i-study.md](#) last revised: ____

Your first week

Everything here is P1 and the start of P2. Nothing else.

Day	Do
1	✓ <i>A Mind for Numbers</i> ch. 1–4. Enroll in <i>Learning How to Learn</i> ; watch week 1. Create /notes with log.md and write the first entry.
2	AMFN ch. 5–8. LHTL week 2. Install Anki; write your first ten cards from what you've read so far. 500 words.
3	AMFN ch. 9–12. LHTL week 3. Take the P3 math test, cold; log the result honestly. 500 words.
4	Watch McEnerney's lecture. Read Keshav's "How to Read a Paper." 500 words on what McEnerney changed for you.
5	Finish AMFN. LHTL week 4. Write how-i-study.md and your weekly template (B.2). 500 words.
6	Missing Semester lecture 1 with ✓ <i>How Linux Works</i> ch. 1 open beside it. Do the exercises. 500 words.
7	Weekly review in the log: what worked, what didn't, what changes next week. Order <i>Make It Stick</i> , <i>How to Read a Book</i> , and <i>Style</i> . Rest.

Then start *Make It Stick* on day 8 and keep going.